Módulo 9: Engenharia de Software Assistida por IA e Sistemas Agentivos (resumo teórico)

Resumo teórico em português para estudo. Cobre o escopo do módulo 9 do edital do TCE-GO (Técnico de Controle Externo, TI): fundamentos da IA aplicada ao desenvolvimento de software; modelos de linguagem de grande porte (LLMs); programação baseada em intenção e desenvolvimento orientado a linguagem natural; engenharia de contexto e de prompts; sistemas agentivos de desenvolvimento (assistentes de codificação em terminal e em IDE, como Claude Code e Antigravity CLI); o ciclo agentivo (descoberta de contexto, planejamento, execução, verificação e autocorreção); geração, revisão, documentação, depuração e testes assistidos por IA; gestão de contexto e memória; RAG e bancos vetoriais; protocolos como o MCP; skills e tool use; avaliação da qualidade do código gerado por IA; alucinações; e segurança, privacidade, propriedade intelectual e ética. Os números citados vêm de laboratórios executados de verdade (L1 a L10) com um modelo local pequeno (Qwen2.5-1.5B-Instruct quantizado, 1,5 bilhão de parâmetros) e uma norma interna fictícia (TI-07); esses resultados ilustram os conceitos e não representam o desempenho dos assistentes de ponta. O que não foi executado está marcado como "não testado". As questões reais de treino vêm de várias bancas (FGV, Cebraspe e FCC), sempre com a fonte, e são completadas por questões autorais marcadas como tal.

1. LLMs: arquitetura, tokens, janela de contexto e parâmetros de geração

- Um LLM (Large Language Model, grande modelo de linguagem) é uma rede neural profunda, em geral um transformer, treinada com enormes volumes de texto para prever o próximo token. Gerar um texto é repetir essa previsão token após token. O modelo não "consulta uma base de fatos": ele produz a continuação mais provável, o que explica tanto a fluência quanto as alucinações.
- Transformer (artigo "Attention Is All You Need", 2017): usa o mecanismo de atenção, em que cada token pondera a relação com os demais tokens do contexto. Ao contrário das redes recorrentes, processa a sequência em paralelo. Pontos que caem em prova: atenção, embeddings de entrada, camadas empilhadas, e a geração autorregressiva (um token por vez).
- Token é um pedaço de texto (palavra, parte de palavra, símbolo). Custo, velocidade e limite de contexto são medidos em tokens, não em caracteres nem em palavras. No laboratório L1: uma frase em português com 56 caracteres e 10 palavras gerou 15 tokens (3,73 caracteres por token); em inglês, 52 caracteres geraram 12 tokens (4,33); uma linha de código com 33 caracteres gerou 12 (2,75); uma sequência de números com 29 caracteres gerou 29 tokens (1,00). Regra prática: texto em português gasta mais tokens por palavra do que em inglês; números e código gastam mais tokens por caractere.
- Janela de contexto é o máximo de tokens que o modelo enxerga de uma vez, somando a entrada e a saída. No L1, um texto de 5000 palavras gerou 5002 tokens e foi recusado por um modelo com janela de 4096 (erro de contexto). Conta de orçamento:

janela de 8000, sistema 500, histórico 2500 e 3 trechos de 900 somam 5700 de entrada; sobram 2300 para a resposta.
- Parâmetros de geração: temperatura controla a aleatoriedade (0 escolhe sempre o token mais provável; valores altos aumentam a variedade e o risco); top-p e top-k limitam o conjunto de candidatos; a semente (seed) permite reproduzir a amostragem; max_tokens limita o tamanho da resposta. No L1, com temperatura 0,0 as 3 execuções deram exatamente a mesma resposta; com temperatura 1,3 e as sementes 1 a 4, as 4 respostas foram diferentes. Para extração e testes, use temperatura baixa; para ideias, mais alta.
- Panorama de modelos: modelos fechados acessados por API (por exemplo, GPT, Claude e Gemini) e modelos de pesos abertos que podem rodar localmente (por exemplo, Llama, Qwen e Mistral). Modelos de embeddings e modelos de linguagem pequenos (SLMs) têm papéis diferentes. Modelos multimodais aceitam imagem e áudio. Não testado: APIs hospedadas; o laboratório usou apenas um modelo local.
- Programação baseada em intenção (Intent-Based Programming) e desenvolvimento orientado a linguagem natural: em vez de escrever cada instrução, a pessoa descreve o resultado desejado (a intenção) em linguagem natural e o sistema gera, executa e ajusta o código. A responsabilidade continua humana: a especificação precisa, os critérios de aceitação e os testes definem o que é "pronto". Sem critérios verificáveis, a intenção fica ambígua e o modelo preenche as lacunas com suposições.
- Contexto é o que o modelo vê em cada chamada: instruções do sistema, histórico, documentos recuperados, resultados de ferramentas e a pergunta. O modelo não tem memória própria entre chamadas; a "memória" é texto que a aplicação reenvia.

2. Treinamento, ajuste fino, RLHF e alucinações

- Fases típicas: pré-treinamento (prever o próximo token em grandes corpora), ajuste por instruções (supervisionado, com exemplos de instrução e resposta) e alinhamento (por exemplo, RLHF, Reinforcement Learning from Human Feedback: avaliadores humanos comparam respostas, treina-se um modelo de recompensa e o LLM é otimizado para respostas úteis e menos nocivas).
- Fine-tuning (ajuste fino) continua o treinamento com dados de um domínio ou de um estilo. Técnicas eficientes como LoRA treinam poucos parâmetros adicionais. Serve para estilo, formato, vocabulário e tarefas repetitivas com dados rotulados. Não é a melhor forma de injetar fatos que mudam: para isso, use RAG. Questão real FGV (MPU 2025) trata de adaptar o BERTimbau a um domínio. Não testado: treinamento, LoRA e RLHF não foram executados no laboratório.
- Quando usar o quê: prompt (rápido, barato, sem treino) > RAG (conhecimento atualizado e privado, com fonte) > fine-tuning (estilo e tarefa estável) > treinar do zero (raramente).
- Alucinação é uma resposta fluente e plausível, porém incorreta ou inventada. Causas: o modelo prevê texto provável, não verifica fatos; lacunas de conhecimento; contexto ruim. No L5, sem recuperação o modelo não acertou 9 de 10 perguntas sobre uma norma que não conhece (errou ou disse que não sabia; por exemplo, inventou "10.000" onde a norma diz 8000 tokens). Mitigações: RAG com fontes, instrução para abster-se

("se não souber, diga que não encontrou"), validação por ferramentas e testes, e revisão humana. Reduzir a temperatura ajuda, mas não elimina.
- Viés e privacidade: o modelo reflete os dados de treino; dados sensíveis enviados em prompts podem ser retidos conforme a política do fornecedor. Detalhes na seção 9.

3. Engenharia de prompt e engenharia de contexto

- Prompt é a entrada que orienta o modelo. Estrutura útil: papel, tarefa, contexto, restrições, formato de saída e exemplos. Prompts claros e específicos reduzem ambiguidade.
- Técnicas: zero-shot (só a instrução), few-shot (alguns exemplos no prompt), chain-of-thought (pedir o raciocínio passo a passo), self-consistency (várias respostas e voto), saída estruturada (JSON com esquema), uso de delimitadores para separar instrução de dado.
- No L2, classificando 12 chamados em dúvida, requisição ou incidente com o modelo pequeno: prompt livre acertou 6 de 12 (50%); zero-shot com instrução e formato, 5 de 12 (42%); few-shot com 3 exemplos, 8 de 12 (67%). A amostra é pequena e o modelo é fraco: o ponto didático é que exemplos costumam ajudar mais do que apenas mais instruções, e que o resultado deve ser medido, não presumido. No mesmo laboratório, o passo a passo não melhorou a aritmética de um modelo tão pequeno (2 de 6 nos dois casos), embora 6 de 6 respostas tenham terminado no formato "Resposta:". Chain-of-thought tende a ajudar mais em modelos maiores; não testado em modelos de ponta.
- Engenharia de contexto é o conjunto de decisões sobre o que entra na janela a cada chamada: instruções do sistema, histórico (resumido ou truncado), documentos recuperados (RAG), definições de ferramentas, resultados de ferramentas e memória de longo prazo. Mais contexto não é sempre melhor: custa tokens, pode diluir o sinal e pode estourar a janela.
- Gestão de contexto e memória em agentes: memória de curto prazo (o histórico da sessão), memória de longo prazo (arquivos de instruções do projeto, notas, índices vetoriais) e estratégias de compactação (resumir o histórico), poda e isolamento por subagentes. Arquivos como AGENTS.md (padrão aberto) ou CLAUDE.md descrevem comandos de build e teste, convenções e restrições do repositório; são contexto em linguagem natural, não um controle técnico de segurança.
- Prompt de sistema versus prompt do usuário: o de sistema define papel e regras; ambos chegam ao modelo como texto e podem ser contornados por injeção (seção 9). Regras críticas devem ser impostas pelo código, não só pelo prompt.

4. Embeddings, busca semântica e bancos vetoriais

- Embedding é um vetor de números que representa um texto de modo que significados próximos fiquem próximos. No L4, o modelo multilíngue paraphrase-multilingual-MiniLM-L12-v2 gerou vetores de 384 dimensões.
- Similaridade do cosseno: $\cos(a, b) = (a \cdot b) / (\|a\| \cdot \|b\|)$, de -1 a 1. Exemplo do L4: o cálculo com numpy e a fórmula manual deram o mesmo valor (0,699164). Distância euclidiana e produto escalar são alternativas; com vetores normalizados o produto escalar equivale ao cosseno.

- Busca lexical (BM25, TF-IDF) depende de termos em comum; busca densa (embeddings) acha paráfrases; a busca híbrida combina as duas, por exemplo com Reciprocal Rank Fusion (RRF). No L4, com 10 perguntas que repetem palavras da norma, BM25, denso e RRF acertaram 10 de 10 na primeira posição; com 4 perguntas parafraseadas (sem palavras em comum), BM25 acertou 0 de 4 e a busca densa, 3 de 4.
- Banco vetorial guarda embeddings e faz busca por vizinhos mais próximos, em geral aproximada (ANN, com índices como HNSW), com filtros por metadados. Exemplos: Qdrant, Milvus, Weaviate, Chroma, pgvector (extensão do PostgreSQL) e FAISS (biblioteca). Não testado: nenhum banco vetorial foi usado no laboratório; o índice foi um arranjo em memória (numpy).
- Cuidados: o modelo de embeddings da consulta deve ser o mesmo da indexação; controle de acesso também vale no índice (um índice compartilhado pode devolver trechos que o usuário não deveria ver: OWASP LLM08); mudar de modelo exige reindexar.

5. RAG: indexação, recuperação, geração e métricas

- RAG (Retrieval-Augmented Generation, geração aumentada por recuperação; artigo de 2020) combina um recuperador e um gerador: busca trechos relevantes em uma base e os insere no prompt para o modelo responder com base neles. Vantagens: conhecimento atualizado sem retreinar, uso de dados privados, redução de alucinação e possibilidade de citar a fonte. Limites: qualidade depende da recuperação; contexto grande custa tokens.
- Indexação: dividir os documentos em trechos (chunking), gerar embeddings e armazenar. Consulta: embutir a pergunta, recuperar os k trechos mais próximos, montar o prompt (instrução + trechos + pergunta) e gerar.
- Chunking: tamanho e sobreposição. Número de trechos = $\lceil (N - \text{tamanho}) / (\text{tamanho} - \text{sobreposição}) \rceil + 1$ (um só trecho se $N \leq \text{tamanho}$). No L3: N = 1000, tamanho 120, sobreposição 20 dá 10; N = 3000 dá 30; N = 500, tamanho 100, sobreposição 0 dá 5; N = 95, tamanho 100, sobreposição 10 dá 1. O código e a fórmula coincidiram nos 4 casos. Trechos muito curtos perdem contexto; muito longos diluem o sinal e enchem a janela. A sobreposição evita cortar uma resposta na fronteira entre dois trechos.
- Efeito do tamanho (L3, 479 palavras, 10 perguntas, k = 2, BM25): 20 palavras (24 trechos) acerto 6/10; 40 (14 trechos) 8/10; 80 (7 trechos) 9/10; 160 (4 trechos) 10/10; 320 (2 trechos) 10/10, mas o contexto médio devolvido passou de 40 para 519 palavras. O simulador de RAG do módulo reproduz esses números.
- Métricas de recuperação: precision@k = relevantes recuperados / k; recall@k = relevantes recuperados / total de relevantes; MRR = média de 1 / posição do primeiro relevante. Aumentar k sobe o recall e tende a baixar a precisão: no L3, k = 1 deu precisão 1,0 e k = 5 deu precisão 0,2, com recall 1,0 nos dois. Exemplo de conta: 4 trechos devolvidos, 2 relevantes, 5 relevantes no total: precision@4 = 0,50 e recall@4 = 0,40.
- Geração: instruir o modelo a responder só com base no contexto e a abster-se quando não encontrar. No L5, sem RAG o modelo acertou 1 de 10 perguntas; com RAG e k = 1, 10 de 10 (151 tokens de entrada em média); com k = 3, 10 de 10 (299 tokens). Em 2

perguntas sem resposta na base, o modelo respondeu "Não encontrei essa informação." (abstenção). Mais trechos custam mais tokens sem ganho aqui.
- Exemplo hipotético do L10 (a questão real TJ-RJ 2025 do módulo afirma que decisões de fragmentação, como tamanho e sobreposição, influenciam a recuperação e a qualidade das respostas): chunk_size = 5000 caracteres e k = 15 recuperam até 75000 caracteres, cerca de 18750 tokens (4 caracteres por token), o que não cabe em uma janela de 4096; chunk_overlap = 0 pode cortar uma resposta na fronteira entre dois trechos. Ajustes: reduzir k ou o tamanho, usar sobreposição, reranqueamento e filtros. Na mesma questão, o item que diz que o RAG reduz o risco de exposição de dados sensíveis porque os documentos sigilosos não precisam ser indexados é falso: o que é recuperado precisa estar indexado, e o índice vira um ativo sensível.
- Outros termos: reranker (reordena os candidatos com um modelo mais preciso), busca híbrida, HyDE, query rewriting, GraphRAG, RAG agentivo (o agente decide quando e o que buscar). Não testado: reranker, GraphRAG e RAG agentivo.

6. Agentes de IA e o ciclo agentivo

- Agente de IA é um LLM com ferramentas, memória e um laço de controle: decide a próxima ação, executa a ferramenta, observa o resultado e repete até concluir ou até atingir um limite. Um workflow é um fluxo predefinido pelo código (o LLM faz passos fixos); um agente decide o fluxo dinamicamente (distinção usada no guia "Building effective agents" da Anthropic).
- Ciclo agentivo de desenvolvimento (escopo do edital): (1) descoberta e coleta de contexto (ler arquivos, buscar no código, ler a documentação); (2) planejamento e tomada de decisão; (3) execução de ações no sistema de arquivos e comandos de shell; (4) verificação autônoma do resultado (rodar testes, lint e build); (5) autocorreção (self-healing): ler o erro, corrigir e tentar de novo. O laço termina por critério de sucesso, limite de passos ou pedido de ajuda humana.
- Padrões: ReAct (raciocínio e ação alternados, com observação a cada passo, artigo de 2022), ReWOO (planeja tudo antes, sem esperar as observações; menos tokens, menos flexível), Plan-and-Execute, reflexão e crítica, multiagente (orquestrador e subagentes especializados, cada um com contexto próprio). Tipos clássicos: reativo simples, baseado em modelo, baseado em objetivos e baseado em utilidade.
- No L6, um agente escrito do zero (laço, ferramentas buscar_norma e calcular, limite de passos) resolveu "descobrir o limite de passos e calcular 6 × 450" em 3 passos: buscar_norma devolveu a norma (limite de 6), calcular('6 * 450') devolveu 2700 e o agente finalizou. Um laço sem fim (calcular('1+1') sem parar) foi contido pelo limite de 6 passos. A ferramenta apagar_repositorio foi bloqueada por estar fora da lista de permissão.
- Com o LLM pequeno decidindo as ações (5 tarefas): prompt básico 2 de 5; com regras e um exemplo no prompt, 4 de 5; com regras e uma guarda contra repetição, 4 de 5. Falhas típicas: nome de ferramenta inválido e repetir a mesma chamada sem concluir. Por isso a guarda e o limite de passos ficam no código.
- Custo em tokens: se o histórico é reenviado a cada passo, o total de entrada é $N \cdot S + A \cdot N(N-1)/2$ (S = pedido inicial, A = tokens acrescentados por passo, N = passos): crescimento quadrático. Exemplo: S = 500, A = 300 e N = 6 dá 3000 + 4500 = 7500

tokens de entrada no total (no L10, as contas com (500, 300, 6), (800, 450, 4) e (200, 100, 10) deram 7500, 5900 e 6500, iguais em Python e em JavaScript). Controles: limite de passos, poda e resumo do histórico, orçamento de tokens.
- Autonomia exige controle: privilégio mínimo, lista de permissão de ferramentas, aprovação humana para ações irreversíveis, modo de planejamento, sandbox, logs e reversibilidade (por exemplo, trabalhar em uma branch e abrir pull request para revisão).

7. MCP, tool use e skills

- Tool use (function calling): o modelo recebe a descrição de funções (nome, parâmetros em JSON Schema) e, quando cabe, devolve uma chamada estruturada (nome e argumentos); a aplicação executa a função e devolve o resultado ao modelo. O modelo não executa nada sozinho.
- MCP (Model Context Protocol) é um padrão aberto para conectar aplicações de IA a ferramentas e dados. Participantes: host (a aplicação de IA, por exemplo uma IDE ou um assistente), cliente MCP (criado pelo host, um por servidor) e servidor MCP (expõe capacidades). A camada de dados usa JSON-RPC 2.0 (requisição com id, method e params; resposta com result ou error); os transportes são stdio (processo local) e HTTP em fluxo (Streamable HTTP, remoto, com OAuth recomendado). O MCP não define como a aplicação usa o LLM.
- Primitivas de um servidor: tools (funções executáveis, que o modelo decide chamar), resources (dados somente leitura por URI, que dão contexto) e prompts (modelos de instrução reutilizáveis). No L7, o servidor "normas-ti", feito com o SDK oficial em Python, ofereceu 2 tools (buscar_norma e contar_trechos), 1 resource (o modelo de URI norma://ti-07/{secao}) e 1 prompt (revisar_pr). A conversa foi feita à mão, em JSON-RPC cru por stdio: tools/list e tools/call (contar_trechos com 3000, 120 e 20 devolveu 30). Uma tool inexistente ou um argumento inválido voltou como erro (isError verdadeiro) sem derrubar o servidor.
- Versões: o laboratório mostrou o ciclo initialize (negociação de versão e capacidades, com a revisão 2025-06-18, seguido da notificação initialized). A revisão 2026-07-28 da especificação, lida na documentação, passou a usar server/discover e a enviar a versão e as informações do cliente em _meta de cada requisição (sem sessão obrigatória), e marcou amostragem e registro como obsoletos; o L7 (seção 6) respondeu a server/discover com a versão 2026-07-28. Mudanças de versão acontecem: confira a documentação.
- Skills e extensibilidade: pacotes reutilizáveis de instruções, scripts e recursos que o agente carrega quando a tarefa pede (por exemplo, um fluxo "revisar pull request" ou "publicar em homologação"); hooks executam comandos antes ou depois de ações (formatar após editar, rodar lint antes do commit); subagentes isolam tarefas; plugins agrupam tudo. Segundo a documentação do Claude Code, o arquivo CLAUDE.md é lido no início de cada sessão e skills empacotam fluxos repetíveis (não testado). Segundo a documentação do Antigravity CLI, há comandos para skills, hooks, MCP, permissões e subagentes (não testado).
- Cuidados: descrições de tools e resultados de servidores de terceiros são entrada não confiável (podem conter injeção); instale somente servidores aprovados; aplique

privilégio mínimo; peça aprovação para ações com efeito externo. Entre as questões reais incluídas não há nenhuma sobre MCP: o tema é coberto por questões autorais, marcadas como tal.
- Interoperabilidade entre agentes: além do MCP (agente e ferramenta), existem propostas para agente e agente (por exemplo, o A2A). Não testado.

8. Assistentes de código: CLI, IDE e o ciclo de desenvolvimento

- Modos de uso: autocompletar em linha; chat com contexto do arquivo; edição guiada em vários arquivos; agente (planeja, edita, executa comandos e testes em laço, em terminal ou IDE); revisor de pull request; agentes em nuvem executados em segundo plano. Quanto maior a autonomia, maior o controle necessário.
- Claude Code (Anthropic), segundo a documentação lida em 28/10/2026: ferramenta de codificação agentiva que lê o código, edita arquivos, executa comandos e se integra a ferramentas; funciona no terminal, em extensões de IDE (VS Code e JetBrains), no aplicativo de desktop e na web; usa CLAUDE.md (ou AGENTS.md) como instruções do projeto, memória automática, skills, hooks, MCP, subagentes e um modo não interativo para CI. Não testado: o Claude Code não foi instalado nem executado neste módulo.
- Antigravity CLI (Google), segundo a documentação lida em 28/10/2026: agente de codificação em terminal, executado com o comando agy; configurações em ~/.gemini/antigravity-cli/settings.json; modos de execução (padrão, accept-edits, auto e plan), gerenciador de permissões (allow, ask e deny), sandbox de terminal, subagentes em segundo plano, MCP, skills, hooks e modo não interativo com saída JSON. Não testado: a ferramenta não foi instalada nem executada neste módulo.
- Equivalentes ou correlatos (nomes e recursos mudam rápido; confira a documentação): GitHub Copilot (modo agente na IDE e agente de código em nuvem), Cursor, Codex CLI, Gemini CLI e outros. Não testado.
- Onde a IA ajuda no ciclo de vida do software (SDLC): requisitos (esclarecer e escrever critérios de aceitação), projeto (alternativas e diagramas), codificação, testes (gerar casos, dados e mocks), revisão de código (apontar defeitos e estilo), depuração (explicar erros e propor correções), documentação (comentários, README, changelog), refatoração e migração, e DevOps (pipelines, scripts).
- Geração de código: é probabilística. No L9, 8 funções pequenas geradas pelo modelo local foram avaliadas por testes automatizados: 4 de 8 passaram de primeira (pass@1 = 50%); falhas: eh_palindromo (SyntaxError), maior_diferenca (ValueError), remover_duplicados e media_ponderada (AssertionError). Com uma retentativa em que a mensagem do teste foi devolvida ao modelo, 5 de 8 passaram (62%): é o self-healing em ação, com ganho limitado. Um fatorial plausível, porém errado, foi barrado pelo teste.
- Geração de testes: a IA propõe casos, mas testes gerados pelo mesmo modelo que gerou o código podem compartilhar o mesmo erro de entendimento; revise a cobertura, os valores esperados e os casos de borda. Teste que "passa" não prova correção se o oráculo estiver errado.
- Boas práticas: dar contexto e critérios de aceitação; pedir um plano antes de editar; trabalhar em passos pequenos e em branch; rodar testes, lint e build a cada passo;

revisar o diff como se fosse de um colega; não colar segredos nem dados pessoais; registrar o que foi gerado por IA conforme a política da organização.
- Riscos: código vulnerável ou desatualizado, dependências inexistentes sugeridas pelo modelo (pacotes alucinados, que atacantes podem registrar), licenças e propriedade intelectual, vazamento de dados no prompt, excesso de confiança (automation bias) e perda de entendimento do sistema.

9. Segurança, privacidade, propriedade intelectual e ética

- OWASP Top 10 para aplicações com LLM, edição 2025: LLM01 Prompt Injection; LLM02 Sensitive Information Disclosure; LLM03 Supply Chain; LLM04 Data and Model Poisoning; LLM05 Improper Output Handling; LLM06 Excessive Agency; LLM07 System Prompt Leakage; LLM08 Vector and Embedding Weaknesses; LLM09 Misinformation; LLM10 Unbounded Consumption.
- Injeção de prompt (LLM01): entrada que altera o comportamento do modelo. Direta (digitada pelo usuário, o "jailbreak") ou indireta (escondida em conteúdo que o modelo lê: página, e-mail, PDF, resultado de ferramenta). O modelo não separa com certeza instrução de dado.
- No L8 (modelo pequeno, 5 ataques em 3 posições do documento, 15 testes por configuração): prompt ingênuo, 3 de 15 ataques bem-sucedidos (início 1 de 5, meio 2 de 5, fim 0 de 5); com o documento delimitado e declarado dado não confiável, 1 de 15; com um filtro de saída de brinquedo (lista de marcadores do teste), 0 de 15. A amostra é pequena e o filtro só conhece os marcadores usados: não é uma defesa real. O teste de sequestro de ferramenta (o agente escolher uma ferramenta perigosa por causa do documento) não ocorreu nos 3 casos testados (2 com documento atacante e 1 de controle), com esse modelo.
- Defesas em camadas: separar instrução de dado; privilégio mínimo e lista de permissão aplicados pelo código; aprovação humana para ações irreversíveis ou com efeito externo; validar e sanitizar a saída antes de usá-la (LLM05); limitar consumo e passos (LLM10); registrar chamadas (log); isolar (sandbox); revisar servidores e plugins de terceiros (LLM03). Nenhum prompt, sozinho, é garantia. No L8, enviar_email sem aprovação foi bloqueado e executou com aprovação humana; apagar_repositorio foi bloqueado mesmo "aprovado", por estar fora da lista.
- Excessive Agency (LLM06): o agente recebe permissões, funções ou autonomia demais. Improper Output Handling (LLM05): tratar a saída do modelo como confiável (por exemplo, executá-la ou montar SQL com ela). Misinformation (LLM09): respostas falsas apresentadas com confiança. Vector and Embedding Weaknesses (LLM08): falhas de acesso e envenenamento em índices vetoriais e embeddings.
- Privacidade e LGPD: dados pessoais enviados a um serviço externo configuram tratamento de dados; é preciso base legal, minimização, política de retenção e contrato com o fornecedor; evitar colar dados pessoais, segredos e documentos sigilosos em assistentes não aprovados. Na norma fictícia do laboratório (TI-07), por exemplo: só o assistente aprovado, só código do repositório interno e revisão humana obrigatória.
- Propriedade intelectual: o código gerado pode reproduzir trechos de código licenciado; é preciso verificar licenças (SCA), políticas internas sobre autoria e uso de código

gerado, e termos do fornecedor. Direitos autorais sobre o resultado variam por jurisdição: consulte o jurídico da organização.
- Ética e governança: transparência, responsabilidade humana (accountability), explicabilidade, não discriminação, supervisão humana e gestão de riscos (por exemplo, o NIST AI RMF). Registrar incidentes e treinar equipes. Na norma fictícia do laboratório: incidente de IA registrado em até 48 horas e treinamento obrigatório de 4 horas, refeito a cada 12 meses.

10. Qualidade e avaliação do código e das respostas de IA

- Avaliar é medir, não presumir. Critérios para código: correção funcional (testes), segurança (SAST, SCA, varredura de segredos), manutenibilidade (legibilidade, complexidade, duplicação), desempenho, aderência ao estilo e à arquitetura.
- O teste automatizado é o juiz objetivo; o assistente não deve avaliar a si mesmo. Revisão humana do diff continua obrigatória. LLM como juiz (outro modelo avalia a resposta) é útil, mas tem vieses e não substitui o teste.
- pass@k: probabilidade de pelo menos uma de k amostras passar nos testes. Estimador: pass@k = 1 − C(n − c, k) / C(n, k), com n amostras geradas e c corretas (artigo do HumanEval/Codex, 2021). Exemplos conferidos no L9 e no L10 (Python e JavaScript iguais): n = 5, c = 2: k = 1 dá 0,40 e k = 2 dá 0,70; n = 10, c = 3: k = 1 dá 0,30 e k = 5 dá 0,9167; n = 5, c = 0 dá 0; n = 5, c = 4, k = 2 dá 1,0.
- Avaliação de RAG e agentes: conjunto de referência (golden set) com perguntas e respostas esperadas, critério de aceite (por exemplo, 90% de acerto em 50 perguntas, como na norma fictícia), métricas de recuperação (precision@k, recall@k, MRR), fidelidade ao contexto (a resposta está sustentada pelos trechos?), taxa de abstenção correta, custo em tokens e latência. Repita a avaliação a cada mudança de prompt, modelo ou índice (regressão).
- Alucinações e validação: peça fontes, confira com ferramentas (testes, busca, execução), use saída estruturada e valide o esquema, e mantenha a revisão humana em decisões de impacto.
- Métricas de negócio e governança: tempo de revisão, defeitos escapados, uso de tokens, custo por tarefa, taxa de aceitação das sugestões. Cuidado com métricas de vaidade (linhas geradas).

11. Resumo para a prova

- Definições: LLM prevê o próximo token; atenção pondera tokens do contexto; token não é palavra; a janela soma entrada e saída; temperatura 0 é (quase) determinística.
- Técnicas: zero-shot, few-shot, chain-of-thought; fine-tuning muda pesos (estilo e tarefa); RAG mantém o conhecimento fora do modelo (fatos atuais e privados, com fonte); RLHF alinha às preferências humanas.
- RAG: chunking (tamanho e sobreposição), embeddings, busca por cosseno, k trechos, prompt, geração; precision@k, recall@k e MRR; BM25 é lexical, a busca densa acha paráfrases, a híbrida combina.
- Agentes: laço pensar, agir e observar; ReAct versus ReWOO; custo quadrático do histórico; limite de passos, lista de permissão e aprovação humana são controles no

código.

- MCP: host, cliente e servidor; JSON-RPC 2.0; stdio e HTTP; tools, resources e prompts; não define o uso do LLM.
- Segurança: injeção de prompt direta e indireta; LLM06 Excessive Agency; LLM05 Improper Output Handling; privilégio mínimo; nenhum prompt é garantia.
- Qualidade: teste automatizado como juiz; pass@k; revisão humana; licenças; LGPD; responsabilidade humana.
- Pegadinhas: absolutos ("sempre", "garante", "elimina", "dispensa") costumam estar errados; confundir caractere, palavra e token; achar que RAG retreina o modelo; achar que o prompt de sistema é um controle de segurança; esquecer que o MCP não escolhe o modelo; confundir pass@1 com pass@k.